CMPE3209 Computer Organization — Proje Teslim Tutanağı

M1 teslimi: Öğrenci bilgileri, Adım 0 ve Adım 1 · 2 Kasım 2026 (Pazartesi)

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Haydar Ali Atmaca
Öğrenci numarası (SID)	5001230016
E-posta	alihaydaratmaca58@gmail.com
Seçilen görev (G1–G5)	G2 — Sınav notu dağılımı
Seçilen ISA ve sürümü	RV32IM
Derleyici ve sürümü	RISC-V (32-bits) gcc 16.1.0 (Compiler Explorer), -march=rv32im -mabi=ilp32 -O0
Simülatör / araçlar	Ripes (ripes.dk web sürümü), ISA RV32IM; Compiler Explorer (godbolt.org)
Git deposu adresi	https://github.com/alihatmaca/Computer-Organization

Adım 0 — Kişiye özel parametreler

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230016 → son hane 6	6
D2	SID'in son iki hanesi	5001230016 → son iki hane 16	16
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+1+6 = 18	18
N	24 + (D2 mod 25)	24 + (16 mod 25) = 24 + 16	40 eleman
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 6 satırı	64 byte / 16 byte line / direct / 4 line / set yok
Write policy	d çift → write-back, d tek → write-through	d = 6 çift	write-back
k	(S mod 7) + 3	(18 mod 7) + 3 = 4 + 3	7
Interrupt türü	S çift → Timer, S tek → I/O	S = 18 çift	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 6	1.6 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × (6 mod 4) = 20 + 5 × 2 = 30	T1 = 1 cycle; T2 = 30 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (18 mod 7) = 2 + 4	6 (varsayım)
Görev parametresi (G2)	P = 50 + (SID mod 10); geçme notu P ve üstü	P = 50 + (5001230016 mod 10) = 50 + 6	P = 56

Not: SID $2^{31} - 1$'i (2 147 483 647) aştığı için programdaki sabit son 9 hane ile yazıldı: const int SID = 1230016;. S, tam numaranın haneleri toplamı olarak alındı; yönerge S için kısaltılmış numarayı belirtmiyor. Kısaltılmış numarayla S = 13 olurdu (k = 9, I/O, Nc = 8).

Adres alanları

Direct mapping, 32 bit adres (RV32): s + w = 32 bit; s = 28 bit.

Alan	Bit sayısı	Hesap
Word (w)	4 bit	log2(16 byte line) = 4
Line (r)	2 bit	log2(4 line) = 2
Tag (s − r)	26 bit	s − r = 28 − 2 = 26

Alan	Bit sayısı	Hesap
Toplam	32 bit	26 + 2 + 4 = 32

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi

☐ G1 Biyomedikal sinyal alarmı
☒ G2 Sınav notu dağılımı
☐ G3 Mesaj şifreleme ve bütünlük
☐ G4 Sıralama ve medyan
☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı

The program makes 40 exam grades from my student number with an LCG (linear congruential generator) and stores them in the global array notlar[40]. Then it goes through the array three times: first it generates the grades, second it counts them into the five grade ranges and adds them up, and third it marks the grades that are below the mean. The function grup() takes a grade and returns the range number (0 to 4) with a chain of if statements. The pass mark is P = 50 + (SID % 10) = 56. The results (bucket[5], mean, below_count and pass_count) are kept in global variables, so I can read them from memory in the simulator. The marking step (| 0x100) is done last, because it makes the grade bigger than 255 and it would break the counting if it was done earlier.

1.3 C kaynak kodu (satır numaralı)

```
 1  /* Haydar Ali Atmaca, 5001230016, G2 */
 2  const int SID = 1230016;
 3  int notlar[40];
 4  int bucket[5];
 5  int mean = 0, below_count = 0, pass_count = 0;
 6
 7  int grup(int g) {
 8      if (g < 50) return 0;
 9      if (g < 65) return 1;
10      if (g < 75) return 2;
11      if (g < 85) return 3;
12      return 4;
13  }
14
15  int main(void) {
16      unsigned int x = SID;
17      int toplam = 0;
18      int P = 50 + (SID % 10);
19
20      for (int i = 0; i < 40; i++) {
21          x = 1664525u * x + 1013904223u;
22          notlar[i] = (x >> 16) % 101;
23      }
24
25      for (int i = 0; i < 40; i++) {
26          toplam += notlar[i];
27          bucket[grup(notlar[i])]++;
28          if (notlar[i] >= P)
29              pass_count++;
30      }
31      mean = toplam / 40;
32
33      for (int i = 0; i < 40; i++) {
34          if (notlar[i] < mean) {
35              notlar[i] = notlar[i] | 0x100;
36              below_count++;
37          }
38      }
39      return 0;
40  }
```

1.4 Zorunlu yapı eşlemesi

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	3; 20, 25, 33	40 elemanlı global notlar dizisi üç döngüyle gezilir: üretim, sayım ve toplama, işaretleme.

#	Zorunlu yapı	Satır no	Açıklama
Y2	Koşullu dallanma	8–11, 28, 34	grup() içindeki if zinciri, geçme notu kontrolü ve ortalama karşılaştırması.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	7 (tanım), 27 (çağrı)	grup(int g) notu alıp 0–4 aralık numarası döndürür; 2. geçişte çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	21, 22, 35	Satır 21 çarpma ve toplama (LCG), satır 22 shift (>> 16) ve mod, satır 35 OR (| 0x100).
Y5	Dizi elemanına store	22, 35 (ayrıca 27)	notlar[i] satır 22 ve 35'te yazılır; bucket[] elemanı satır 27'de artırılır.
Y6	const int SID ve hesapta kullanımı	2; 16, 18	SID, LCG'nin başlangıç değeridir (satır 16) ve geçme notu P = 50 + (SID % 10) hesabında kullanılır (satır 18).
Y7	Sonucun tutulduğu global değişken(ler)	4–5	bucket[5], mean, below_count ve pass_count global değişkenlerdir.

1.5 Program çıktıları

Program Ripes'te çalıştırılıp çıkış kodu 0 ile bittikten sonra Memory sekmesinden okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
bucket[0]	0x100000A4	16 (0x10)	m1_bellek.png
bucket[1]	0x100000A8	9 (0x09)	m1_bellek.png
bucket[2]	0x100000AC	3 (0x03)	m1_bellek.png
bucket[3]	0x100000B0	4 (0x04)	m1_bellek.png
bucket[4]	0x100000B4	8 (0x08)	m1_bellek.png
mean	0x100000B8	56 (0x38)	m1_bellek.png
below_count	0x100000BC	18 (0x12)	m1_bellek.png
pass_count	0x100000C0	22 (0x16)	m1_bellek.png

Kontrol: bucket toplamı 16 + 9 + 3 + 4 + 8 = 40 = N; below_count + pass_count = 18 + 22 = 40.

1.6 İzlenecek kernel

Alan	Değer
C satır aralığı	8, 13 ve 27 (grup() içinde if (g < 50) return 0 ve fonksiyon çıkışı; ardından bucket[grup(notlar[i])]++)
Başlangıç ve bitiş adresi (hex)	0x28 (bgt a4,a5,.L2) — `0x1e8` (sw a4,0(a5))
Dinamik instruction sayısı	20 (la pseudo-instruction'ı auipc + addi olarak 2 sayıldı)
Hangi döngü iterasyonu	2. geçiş, i = 3 (notlar[3] = 19 → grup() 0 döndürür)

Dinamik instruction sırası (RISC-V gcc 16.1.0, -O0):

#	Instruction	Açıklama
1	bgt a4,a5,.L2	Koşullu branch, alınmaz (g = 19 ≤ 49)
2	li a5,0	Sabit yükleme (ALU)
3	j .L3	Koşulsuz atlama
4	mv a0,a5	Dönüş değerini a0'a koy
5	lw ra,28(sp)	Load
6	lw s0,24(sp)	Load
7	addi sp,sp,32	ALU

#	Instruction	Açıklama
8	jr ra	Return
9	mv a5,a0	Sonucu al
10–11	la a3,bucket	auipc + addi
12	slli a4,a5,2	ALU (shift)
13	add a4,a3,a4	ALU
14	lw a4,0(a4)	Load (bucket[0])
15	addi a4,a4,1	ALU
16–17	la a3,bucket	auipc + addi
18	slli a5,a5,2	ALU (shift)
19	add a5,a3,a5	ALU
20	sw a4,0(a5)	Store (dizi elemanı)

Kernel load, store, ALU, koşullu branch ve return içerir. Pencere bellekte ardışık değildir (j .L3 ve jr ra atlar); bu yüzden başlangıç ve bitiş adresi arasındaki fark 20 × 4 byte değildir. Programı Ripes'te çalıştırmak için GCC çıktısının başına 3 satırlık bir başlangıç şablonu (jal ra,main; li a7,10; ecall) eklendi; kernel bundan etkilenmez.

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları / işlemleri	Adres aralığı (hex)
7	int grup(int g)	addi sp; sw ra/s0; addi s0; sw a0	0x0000000C–0x0000001C
8	if (g < 50) return 0	lw; li 49; blt; li 0; j	0x00000020–0x00000030
9	if (g < 65) return 1	lw; li 64; blt; li 1; j	0x00000034–0x00000044
10	if (g < 75) return 2	lw; li 74; blt; li 2; j	0x00000048–0x00000058
11–13	if (g < 85)...; return 4; çıkış	lw; li; blt; li; j; mv; lw ra/s0; addi; ret	0x0000005C–0x00000084
15–17	main() başlangıcı; x=SID; toplam=0	addi sp; sw; lui; addi; sw zero	0x00000088–0x000000A4
18	P=50+(SID % 10)	lui/addi; mulh; srai; sub; slli; add; sw	0x000000A8–0x000000E0
20	for (i=0;i<40;i++)	sw zero; j; lw; addi; sw; bge	0x000000E4–0x000000E8, 0x00000158–0x0000016C
21	LCG: x=1664525*x+1013904223	lw; lui; addi; mul; add; sw	0x000000EC–0x00000108
22	notlar[i]=(x>>16)%101	lw; srli; mulhu; sub; add; la; slli; sw	0x0000010C–0x00000154
25	ikinci for döngüsü	sw zero; j; lw; addi; sw; bge	0x00000170–0x00000174, 0x00000228–0x0000023C
26	toplam += notlar[i]	la; lw; slli; add; lw; add; sw	0x00000178–0x00000198
27	bucket[grup(notlar[i])]++	la; lw; mv; jal; slli; add; lw; addi; sw	0x0000019C–0x000001E8
28–29	if (notlar[i]>=P) pass_count++	la; lw; blt; lw; addi; sw	0x000001EC–0x00000224
31	mean = toplam / 40	lw; lui/addi; mulh; srai; sub; sw	0x00000240–0x00000264
33–34	üçüncü for; if (notlar[i]<mean)	lw; slli; add; lw; bge	0x00000268–0x00000294, 0x000002E8–0x000002FC
35	notlar[i] |= 0x100	lw; ori; la; slli; add; sw	0x00000298–0x000002C8
36	below_count++	la; lw; addi; sw	0x000002CC–0x000002E4
39–40	return 0; main çıkışı	li; mv; lw ra/s0; addi sp; ret	0x00000300–0x00000314

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00000000	792 byte (0x318)	Başlangıç şablonu, grup ve main instruction'ları
.data	0x10000000	196 byte (0xC4)	SID, notlar[40], bucket[5], mean, below_count, pass_count

Von Neumann ilkesi: Programın instruction'ları .text, global değişkenleri .data bölümündedir. Farklı başlangıç adreslerine sahip olsalar da aynı 32-bit adres uzayında tutulurlar.

Not: Adresler Ripes için düzenlenen -O0 assembly'nin .text=0x00000000 ve .data=0x10000000 yerleşimiyle alınmıştır; C satır eşlemesi temel blokları gösterir.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel instruction sayısı	20	11	Gereksiz stack / dönüş ve adres hesabı azaldı
Load sayısı	3	1	-O0: ra, s0, bucket; -O1: bucket
Store sayısı	1	1	İki sürüm de bucket elemanını yazar
Kullanılan register sayısı	7	6	-O1, stack/return register erişimlerini kaldırır; a4,a0,a5,s4,s6,s9 kullanılır

Sayım yöntemi: O0 = 20, O1 = 11 dinamik instruction. O0 yolu 3 load / 1 store / 7 register; O1 yolu 1 load / 1 store / 6 register içerir. Fonksiyonun girişindeki hazırlık ve call komutu bu karşılaştırma penceresine dahil değildir.

Karşılaştırma: ikinci geçişte i = 3, notlar[3] = 19 için grup() sonucundan bucket[0]++ yazılmasına kadar yürütülen dinamik instruction yolu. Register sayısı, bu yol üzerinde kullanılan farklı register adlarının sayısıdır (zero hariç).

2c. Elle machine code (12 instruction; R, I, S, B, U, J formatları):

#	Adres	Assembly	Format	Machine code	Byte 0 → 3	objdump ile aynı mı?
1	0x00000098	lui a5,0x12c	U	0x0012C7B7	B7 C7 12 00	Evet
2	0x0000009C	addi a5,a5,1216	I	0x4C078793	93 87 07 4C	Evet
3	0x00000028	blt a5,a4,.L2	B	0x00E7C663	63 C6 E7 00	Evet
4	0x00000020	lw a4,-20(s0)	I	0xFEC42703	03 27 C4 FE	Evet
5	0x000001B8	jal ra,grup	J	0xE55FF0EF	EF F0 5F E5	Evet
6	0x000001CC	add a4,a3,a4	R	0x00E68733	33 87 E6 00	Evet

#	Adres	Assembly	Format	Machine code	Byte 0 → 3	objdump ile aynı mı?
7	0x000001D0	lw a4,0(a4)	I	0x00072703	03 27 07 00	Evet
8	0x000001D4	addi a4,a4,1	I	0x00170713	13 07 17 00	Evet
9	0x000001E0	slli a5,a5,2	I	0x00279793	93 97 27 00	Evet
10	0x000001E8	sw a4,0(a5)	S	0x00E7A023	23 A0 E7 00	Evet
11	0x00000000	jal ra,main	J	0x088000EF	EF 00 80 08	Evet
12	0x000002B0	ori a4,a5,256	I	0x1007E713	13 E7 07 10	Evet

Zorunlu instruction sıra numaraları: SID yükleme #1–2; koşullu branch #3; load #4, #7; store #10; fonksiyon çağrısı #5 (ayrıca başlangıç çağrısı #11). Objdump karşılaştırması: listelenen 12 instruction için aynı.

Bit alanları (ad = binary; formatın bit sırasıyla):

# / Format	Alanlar (binary)
1	1 / U: imm[31:12]=00000000000100101100 | rd=01111 | opcode=0110111
2	2 / I: imm[11:0]=010011000000 | rs1=01111 | funct3=000 | rd=01111 | opcode=0010011
3	3 / B: imm[12]=0 | imm[10:5]=000000 | rs2=01110 | rs1=01111 | funct3=100 | imm[4:1]=0110 | imm[11]=0 | opcode=1100011
4	4 / I: imm[11:0]=111111101100 | rs1=01000 | funct3=010 | rd=01110 | opcode=0000011
5	5 / J: imm[20]=1 | imm[10:1]=1100101010 | imm[11]=1 | imm[19:12]=11111111 | rd=00001 | opcode=1101111
6	6 / R: funct7=0000000 | rs2=01110 | rs1=01101 | funct3=000 | rd=01110 | opcode=0110011
7	7 / I: imm[11:0]=000000000000 | rs1=01110 | funct3=010 | rd=01110 | opcode=0000011
8	8 / I: imm[11:0]=000000000001 | rs1=01110 | funct3=000 | rd=01110 | opcode=0010011
9	9 / I: imm[11:0]=000000000010 | rs1=01111 | funct3=001 | rd=01111 | opcode=0010011
10	10 / S: imm[11:5]=0000000 | rs2=01110 | rs1=01111 | funct3=010 | imm[4:0]=00000 | opcode=0100011
11	11 / J: imm[20]=0 | imm[10:1]=0001000100 | imm[11]=0 | imm[19:12]=00000000 | rd=00001 | opcode=1101111
12	12 / I: imm[11:0]=000100000000 | rs1=01111 | funct3=110 | rd=01110 | opcode=0010011

SID’in yüklenmesi

Tam SID, signed 32 bit aralığını aştığı için yönergedeki son 9 hane kullanılır: 1230016 = 0x0012C4C0. lui a5,0x12C → 0x0012C000; addi a5,a5,1216 (0x4C0) → 0x0012C4C0. ADDI immediate alanı signed 12 bit olduğundan sabit tek instruction’a sığmaz. Alt 12 bit 0x800 veya üstü olsaydı sign extension yüzünden üst 20 bit için +1 düzeltmesi gerekirdi; burada 0x4C0 olduğundan gerekmez.

Branch PC-relative offset hesabı

Branch PC = 0x00000028; hedef .L2 = 0x00000034. Offset = 0x34 − 0x28 = 0x0C = +12 byte. Gerçek instruction: blt a5,a4,.L2 (GCC metnindeki bgt a4,a5,.L2 pseudo-instruction’ının karşılığı). B-format alanları 2c tablosundaki #3 satırında gösterildi. Kod = 0x00E7C663; little-endian byte sırası = 63 C6 E7 00.

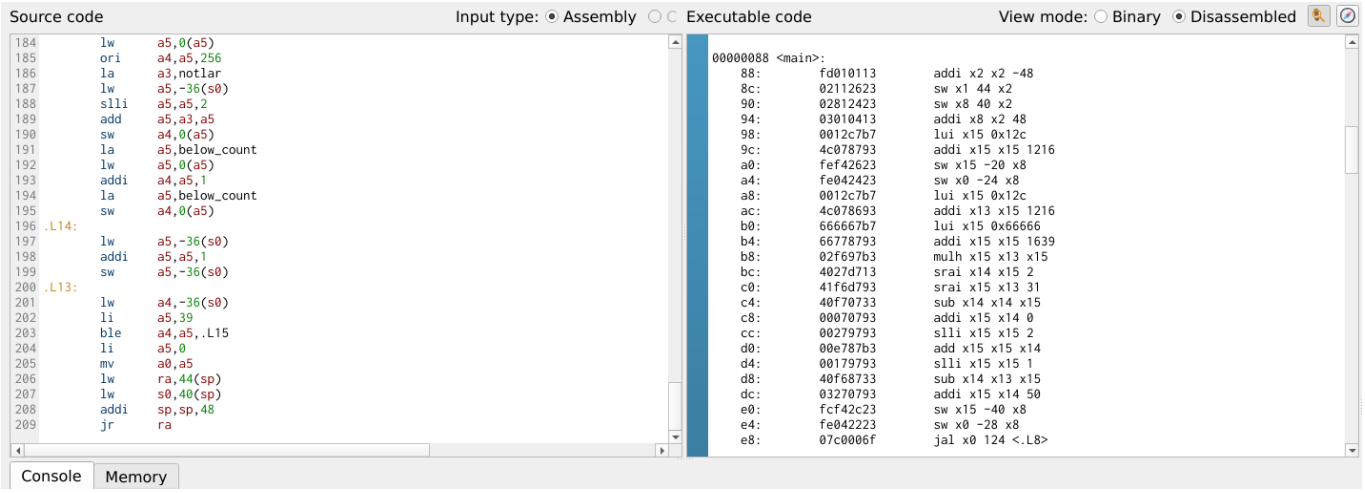
Doğrulama / dosya atıfları

Tam assembly: asm_O0.s ve asm_O1.s. Machine code: objdump_O0.txt, objdump_O1.txt ve Asama2_machine_code_tablosu.csv. Ripes’te gözlenen SID kodu ve branch kodu bu tablodaki değerlerle uyuşmaktadır. M1 1.6 tablosundaki kernel son adresi 0x1E8 olarak düzeltilmiştir. 0x1EC sonraki auipc instruction adresidir.

Adım 2 — Machine code doğrulama ekran görüntüleri

Ripes RV32IM: Aşağıdaki görüntülerde disassembly adresi, hexadecimal machine code ve instruction birlikte görülür. Tam doğrulama listesi objdump_O0.txt dosyasındadır.

Kanıt 1 — main, SID sabiti: 0x98 lui ve 0x9C addi



Kanıt 2 — grup, branch: 0x28 blt, hedef 0x34

